

EXPERIMENT 1

Predictive Text System using N-Gram and HMM

AIM:

To develop a predictive text system using N-Gram language models and Hidden Markov Models (HMM) to predict the next word based on previous context using the Brown Corpus dataset.

PROCEDURE:

1. Import required libraries: nltk, collections (defaultdict, Counter), and random.
2. Download the Brown Corpus using nltk.download('brown').
3. Load and lowercase all words from the corpus to create a word list.
4. Generate bigrams (pairs of consecutive words) from the word list.
5. Count frequencies of bigrams and unigrams using Counter.
6. Build a bigram probability model using defaultdict.
7. For each (w1, w2) pair in bigram counts, compute conditional probability: $P(w2|w1) = \text{count}(w1, w2) / \text{count}(w1)$.
8. Define predict_next_word(word): return the word with highest bigram probability given the input word.
9. For HMM-based prediction: load POS-tagged Brown Corpus data.
10. Train transition probabilities (tag $\rightarrow$ next tag) and emission probabilities (tag $\rightarrow$ word).
11. Define predict_next(prev_tag): find the most likely next tag and corresponding word.
12. Test predictions on sample inputs and print results.

CODE:

```
import nltk
from nltk.corpus import brown
from collections import defaultdict, Counter
import random

nltk.download('brown')
words = [w.lower() for w in brown.words()]
bigrams = [(words[i], words[i+1]) for i in range(len(words)-1)]
bigram_counts = Counter(bigrams)
unigram_counts = Counter(words)
bigram_prob = defaultdict(dict)

for (w1, w2), count in bigram_counts.items():
    bigram_prob[w1][w2] = count / unigram_counts[w1]

def predict_next_word(word):
    if word in bigram_prob:
        return max(bigram_prob[word], key=bigram_prob[word].get)
    else:
        return random.choice(words)

input_word = 'the'
print('Input Word:', input_word)
print('Predicted Next Word:', predict_next_word(input_word))

# HMM-based prediction
nltk.download('universal_tagset')
data = brown.tagged_sents(tagset='universal')
transition = defaultdict(Counter)
emission = defaultdict(Counter)
```

```
for sent in data:
    prev = '<START>'
    for word, tag in sent:
        word = word.lower()
        transition[prev][tag] += 1
        emission[tag][word] += 1
        prev = tag

def predict_next(prev_tag):
    next_tag = max(transition[prev_tag], key=transition[prev_tag].get)
    next_word = max(emission[next_tag], key=emission[next_tag].get)
    return next_word, next_tag

word, tag = predict_next('DET')
print('Previous POS Tag: DET')
print('Predicted Word:', word)
print('Predicted POS Tag:', tag)
```

OUTPUT:

[nltk_data] Downloading package brown to /root/nltk_data...

[nltk_data] Unzipping corpora/brown.zip.

Input Word: the

Predicted Next Word: first

[nltk_data] Downloading package brown to /root/nltk_data...

[nltk_data] Package brown is already up-to-date!

[nltk_data] Downloading package universal_tagset to /root/nltk_data...

[nltk_data] Package universal_tagset is already up-to-date!

Previous POS Tag: DET

Predicted Word: time

Predicted POS Tag: NOUN

RESULT:

The predictive text system was successfully developed using N-Gram (Bigram) language model and Hidden Markov Model. The bigram model predicted 'first' as the next word after 'the'. The HMM model predicted 'time' (NOUN) following a determiner (DET) tag, demonstrating successful next-word prediction using probabilistic language models on the Brown Corpus.